

技术设计方案 · TECHNICAL DESIGN

全平台本地 阅读App

面向 iOS、Android、HarmonyOS 及桌面端的全平台本地阅读应用设计方案，核心聚焦于书源抽象引擎与多源切换机制，支持 TXT、EPUB、PDF 等多种格式的本地与在线混合阅读体验。

星尘 工程设计

2026年6月

CROSS-PLATFORM READING APP DESIGN

目录

第一章 项目概述	4
1.1 产品定位	4
1.2 核心功能	4
1.3 目标用户	4
第二章 技术选型	6
2.1 跨平台框架对比	6
2.2 推荐方案：Flutter	6
2.3 技术栈总览	7
第三章 系统架构设计	9
3.1 分层架构	9
3.2 模块划分	10
3.3 依赖关系	10
第四章 换源核心机制设计	12
4.1 设计目标	12
4.2 书源抽象模型	12
4.2.1 BookSource 接口定义	12
4.2.2 书源数据结构	13
4.3 换源流程	14
4.3.1 书源匹配算法	14
4.3.2 章节映射算法	15
4.4 书源格式设计	16
4.5 多源管理策略	18
第五章 数据模型设计	19
5.1 核心实体关系	19
5.2 数据库表结构	19

5.2.1 书籍表 (books)	19
5.2.2 书源关联表 (book_sources)	20
5.2.3 章节表 (chapters)	20
5.2.4 阅读进度表 (read_progress)	21
5.2.5 书签与笔记表	21
5.3 章节缓存策略	22
第六章 本地文件与存储方案	23
6.1 文件格式支持	23
6.2 TXT 文件解析	23
6.3 EPUB 解析	24
6.4 本地存储架构	24
第七章 阅读引擎设计	25
7.1 阅读模式	25
7.2 分页引擎	25
7.3 排版引擎	26
7.4 主题系统	27
第八章 UI/UX设计	28
8.1 页面结构	28
8.2 阅读页交互设计	28
8.3 换源交互流程	29
8.4 搜索体验设计	29
第九章 网络层与书源引擎	30
9.1 书源执行引擎	30
9.2 并发控制	31
9.3 反爬策略	32
9.4 错误处理与降级	32
第十章 实现路线图	33

10.1 阶段划分	33
10.2 MVP 阶段（第1-6周）	33
10.3 V1.0 阶段（第7-14周）	33
10.4 V2.0 阶段（第15-22周）	33
10.5 风险与应对	34

第一章 项目概述

1.1 产品定位

本项目旨在设计一款面向全平台（iOS、Android、HarmonyOS、Windows、macOS、Linux）的本地优先阅读应用。应用以"本地阅读为核心、书源扩展为增强"为核心理念，用户既可以将本地 TXT、EPUB、PDF 等文件导入应用进行沉浸式阅读，也可以通过配置在线书源来搜索、发现和获取网络书籍资源，实现本地与在线阅读体验的无缝融合。产品的差异化竞争力在于强大的换源机制——当某个书源失效或内容质量下降时，用户可以一键切换到其他书源继续阅读，无需重新搜索，阅读进度自动衔接。

1.2 核心功能

应用的功能体系围绕"阅读体验"和"书源生态"两条主线展开，涵盖从书籍获取、管理、阅读到同步的完整闭环。以下表格列出了应用的核心功能模块及其在设计中的优先级。

功能模块	描述	优先级
本地书导入	支持 TXT / EPUB / PDF / MOBI 格式导入与解析	P0
书架管理	书籍分组、排序、搜索、批量操作	P0
阅读引擎	分页/滚动模式、排版引擎、字体/行距/边距调节	P0
书源系统	书源导入、管理、搜索、详情获取	P0
换源机制	同一书籍多书源切换、内容对比、自动换源	P0
阅读进度	章节进度、页内位置、跨设备同步	P1
主题系统	日间/夜间/护眼/自定义主题	P1
书签笔记	书签标记、划线、笔记、导出	P1
TTS朗读	文本转语音、自动翻页	P2
云同步	阅读进度、书源、书签的云端同步	P2

表 1-1 核心功能模块与优先级

1.3 目标用户

本应用的目标用户群体主要包括以下几类。首先是网络文学爱好者，他们习惯于从多个阅读平台获取内容，希望有一个统一的工具来管理所有书籍，并且当某个来源不可用时能够快速切换到其他来源。其次是本地阅读用户，他们拥有大量电子书文件，需要一个功能强大的阅读器来管理和阅读这些文件。第三类是注重隐私的用户，他们不希望阅读数据被上传到云端，倾向于完全本地的阅读方案，同时保留对在线资源的可选访问能力。最后是跨设备用户，他们在手机、平板和桌面端之间切换阅读，需要一致的用户体验和同步能力。

第二章 技术选型

2.1 跨平台框架对比

全平台阅读应用的技术选型首要解决的是跨平台框架的选择。我们对当前主流的三种跨平台方案进行了系统对比，从性能、生态、开发效率、维护成本等维度进行综合评估。

维度	Flutter	React Native	Kotlin Multiplatform
语言	Dart	JavaScript/TypeScript	Kotlin
UI渲染	自绘引擎 (Skia/Impeller)	原生组件映射	Compose Multiplatform
性能	接近原生 (60-120fps)	接近原生 (JS Bridge开销)	原生级别
生态	pub.dev, 快速增长	npm, 最丰富	初阶段, 快速成长
桌面端	官方支持 (Windows/macOS/Linux)	社区方案, 不成熟	官方支持 (Compose Desktop)
HarmonyOS	社区支持, 待完善	不支持	不支持
学习曲线	中等 (Dart语言)	低 (JS生态)	中高 (Kotlin + Compose)
包体积	较大 (~15MB baseline)	中等 (~10MB)	较小 (~8MB)
热重载	支持	支持 (Fast Refresh)	支持 (Compose Preview)

表 2-1 跨平台框架多维度对比

2.2 推荐方案：Flutter

综合评估后，本设计推荐采用 Flutter 作为主要开发框架。选择 Flutter 的核心理由如下。

真正的全平台覆盖。Flutter 是唯一一个在移动端（iOS/Android）、桌面端（Windows/macOS/Linux）和嵌入式端都有官方支持方案的跨平台框架。虽然 HarmonyOS 目前仅有社区支持，但鸿蒙官方已推出 Flutter 鸿蒙适配项目，预计在 2026 年内达到稳定状态。这意味着一套代码可以覆盖所有目标平台，大幅降低开发和维护成本。

自绘引擎保证一致的阅读体验。阅读应用对排版的精确度要求极高——字体渲染、行间距、分页算法、段落间距都需要像素级的控制。Flutter 的 Skia/Impeller 自绘引擎不依赖平台原生组件，可以确保在不同平台上呈现完全一致的阅读界面，这对于阅读类应用是决定性优势。React Native 依赖原生组件映射，各平台差异难以统一；而 Kotlin Multiplatform 的 Compose Desktop 在桌面端尚处于早期阶段，排版能力不如 Flutter 成熟。

丰富的文本处理生态。Flutter 生态中已有 `super_text`、`flutter_widget_from_html_core` 等高质量文本排版库，以及对 EPUB 解析、PDF 渲染的成熟方案。Dart 语言的 `isolate` 模型适合处理大文件解析和网络请求等耗时操作，不会阻塞 UI 线程。

性能满足阅读场景需求。Flutter 的渲染性能接近原生，在分页动画、滚动流畅度、字体加载等关键指标上表现优异。对于阅读应用而言，60fps 的滚动和翻页动画完全可以由 Flutter 承载，无需为了极致性能而选择原生开发。

2.3 技术栈总览

层级	技术选型	说明
框架	Flutter 3.x	全平台 UI 框架
语言	Dart 3.x	空安全、模式匹配
状态管理	Riverpod 2.x	类型安全、可测试、编译时检查
路由	go_router	声明式路由、深链接支持
本地数据库	Drift (SQLite)	类型安全 ORM、响应式查询
KV存储	SharedPreferences / Hive	轻量配置存储
网络请求	Dio + Retrofit	拦截器、取消、并发控制
HTML解析	html package	书源页面解析
EPUB解析	epubx / 自研解析器	EPUB 2/3 标准
PDF渲染	syncfusion_flutter_pdfviewer	高性能 PDF 渲染
依赖注入	Riverpod Provider	与状态管理统一
序列化	json_serializable / freezed	不可变模型、JSON 序列化

层级	技术选型	说明
日志	logger	分级日志、文件输出

表 2-2 技术栈总览

第三章 系统架构设计

3.1 分层架构

应用采用经典的四层分层架构，各层职责明确、单向依赖，通过依赖注入实现解耦。架构设计遵循关注点分离原则，确保每一层都可以独立测试和演进。

层级	职责	主要组件
表现层 (Presentation)	UI渲染、用户交互、状态展示	页面Widget、Riverpod Provider、路由
领域层 (Domain)	业务逻辑、领域模型、用例	BookService、SourceService、ReadEngine
数据层 (Data)	数据访问、缓存、仓库模式	Repository、DAO、APIClient
平台层 (Platform)	平台能力封装、文件系统、数据库	Database、FileSystem、PlatformChannel

表 3-1 四层架构职责划分

表现层负责将业务状态映射为用户可见的界面，包含所有页面 Widget 和对应的 Riverpod Provider。页面 Widget 仅负责渲染和事件转发，不包含业务逻辑。每个页面关联一个或多个 Provider，Provider 持有 UI 状态（如加载中、数据、错误）并监听领域层的状态变化。

领域层是应用的核心，包含所有业务规则和用例编排。这一层不依赖任何具体的数据访问实现，通过 Repository 接口与数据层交互。领域层定义了 Book、BookSource、Chapter 等核心领域模型，以及 BookService（书籍管理）、SourceService（书源管理）、ReadEngine（阅读引擎）等领域服务。换源机制的核心逻辑——书源匹配、内容获取、进度映射——全部位于此层。

数据层实现了 Repository 模式，为领域层提供统一的数据访问接口。每个 Repository 封装了本地数据源（SQLite、文件系统）和远程数据源（网络 API）的访问逻辑，并负责缓存策略。例如 BookRepository 同时管理本地导入的书籍和通过书源获取的在线书籍，对领域层透明。

平台层封装了与操作系统交互的底层能力，包括 SQLite 数据库（通过 Drift）、文件系统访问、平台特有的功能（如 iOS 的 UIDocumentPicker、Android 的 Storage Access Framework）。这一层隔离了平台差异，使上层代码无需关心具体的平台实现。

3.2 模块划分

在四层架构的基础上，应用按功能职责划分为以下核心模块。每个模块是一个独立的 Feature Package，拥有自己的领域模型、数据访问和 UI 页面，模块之间通过定义良好的接口通信。

模块	功能描述	核心类
bookshelf	书架管理、书籍CRUD、分组排序	BookshelfController, BookRepository
reader	阅读界面、分页引擎、排版渲染	ReaderController, PaginationEngine, TextLayout
source	书源管理、书源导入、书源编辑	SourceManager, SourceImporter, SourceValidator
engine	书源执行引擎、搜索、详情获取	SourceEngine, SearchTask, ChapterFetcher
switcher	换源核心逻辑、书源匹配、进度映射	SourceSwitcher, SourceMatcher, ProgressMapper
import	本地文件导入、格式解析	ImportService, EpubParser, TxtParser, PdfParser
library	数据库、缓存、文件管理	Database, CacheManager, FileManager
sync	云同步、进度同步、书源同步	SyncManager, SyncEngine
theme	主题管理、阅读主题、夜间模式	ThemeManager, ReaderTheme

表 3-2 核心模块划分

3.3 依赖关系

模块之间的依赖关系遵循单向无环原则（DAG）。核心依赖链路如下：bookshelf 依赖 import 和 library；reader 依赖 engine（用于在线章节获取）和 library（用于本地章节读取）；engine 依赖 source（书源定义）和 library（缓存）；switcher 依赖 engine（获取各书源章节列表）和 reader（进度映射）；sync 依赖 library 和 source。所有模块共享 library 模块的基础设施能力。

这种设计确保了换源模块（switcher）可以作为独立关注点存在，它不修改书籍本身的阅读状态，而是在底层切换数据来源，对 reader 模块透明。当用户触发换源时，switcher 负责协调 engine 获取新书源的章节列表，通过 ProgressMapper 将当前阅读位置映射到新书源的

对应章节，然后通知 reader 重新加载内容。

第四章 换源核心机制设计

4.1 设计目标

换源是本应用的核心差异化功能。其设计需要达成以下目标。第一，用户在阅读过程中遇到内容缺失、章节错乱、更新延迟等问题时，能够一键切换到其他可用书源，切换过程对用户透明，阅读进度自动衔接。第二，系统应能自动检测书源可用性，在书源失效时主动提示用户换源，甚至自动切换到最优书源。第三，换源操作不应破坏用户的阅读上下文（书签、笔记、阅读位置），这些数据需要在书源切换后保持有效。第四，换源机制需要支持"多源并行"模式——同一本书可以同时关联多个书源，系统自动选择内容最完整、更新最及时的来源。

4.2 书源抽象模型

换源机制的基础是一套统一的书源抽象协议。所有书源——无论是内置的、用户导入的、还是社区分享的——都必须实现这套协议。协议定义了搜索、详情获取、目录获取、章节内容获取四个核心操作，以及书源元信息（名称、URL、分组、排序权重）。

4.2.1 BookSource 接口定义

以下是书源协议的 Dart 接口定义，所有书源必须实现此接口。协议采用异步设计，所有网络操作返回 Future，支持超时取消。

```
/// 书源协议接口 — 所有书源必须实现
abstract interface class BookSource {
  /// 书源唯一标识
  String get id;

  /// 书源名称（显示用）
  String get name;

  /// 书源基础URL
  String get baseUrl;

  /// 书源分组（用于管理和筛选）
  String get group;

  /// 权重（多源排序用，数值越大优先级越高）
  int get weight;

  /// 是否启用
  bool get enabled;

  /// 搜索书籍
  /// [keyword] 搜索关键词
  /// [page] 分页页码（从1开始）
```

```

/// 返回搜索结果列表
Future< search({
  required String keyword,
  int page = 1,
});

/// 获取书籍详情
/// [bookKey] 书源内的书籍标识
Future getDetail({required String bookKey});

/// 获取章节目录
/// [bookKey] 书源内的书籍标识
Future< getChapterList({
  required String bookKey,
});

/// 获取章节内容
/// [bookKey] 书源内的书籍标识
/// [chapterKey] 章节标识
Future getChapterContent({
  required String bookKey,
  required String chapterKey,
});

/// 探索发现（可选实现）
Future< discover({int page = 1});

/// 健康检查
Future healthCheck();
}

```

4.2.2 书源数据结构

书源搜索结果、书籍详情、章节信息等数据结构的定义如下。这些结构同时用于本地书籍和在线书源，确保换源时数据格式统一。

```

/// 搜索结果
class SearchResult {
  final String bookKey; // 书源内唯一标识
  final String title; // 书名
  final String author; // 作者
  final String? coverUrl; // 封面URL
  final String? intro; // 简介
  final String? category; // 分类
  final String sourceId; // 来源书源ID
}

/// 书籍详情
class BookDetail {
  final String bookKey;

```

```

final String title;
final String author;
final String? coverUrl;
final String? intro;
final String? category;
final int? wordCount; // 字数
final String? status; // 连载中/已完结
final String? lastChapter; // 最新章节
final String sourceId;
}

/// 章节信息
class ChapterInfo {
final String chapterKey; // 章节唯一标识
final String title; // 章节标题
final int order; // 章节序号
final bool isVip; // 是否VIP章节
final bool isPaid; // 是否已付费
}

/// 章节内容
class ChapterContent {
final String chapterKey;
final String title;
final String content; // 正文文本（已清洗）
final String sourceId;
final DateTime fetchedAt; // 获取时间
}

```

4.3 换源流程

换源流程是整个机制的核心。当用户在阅读过程中触发换源（手动或自动），系统需要完成以下步骤。首先是书源匹配——系统在所有已启用的书源中搜索与当前书籍匹配的结果。匹配策略采用"书名 + 作者"双重匹配，辅以模糊匹配容错。其次是章节映射——将当前阅读位置（章节序号）映射到新书源的章节列表。由于不同书源的章节划分可能不同（如合并章节、拆分章节），映射算法需要处理章节标题相似度匹配。最后是内容切换——从新书源加载对应章节内容，无缝替换当前阅读内容，保持页内位置（如可能）。

4.3.1 书源匹配算法

书源匹配是换源的第一步，目标是在所有已启用书源中找到与当前书籍对应的条目。匹配算法采用多级筛选策略，逐步缩小候选集，兼顾准确性和效率。

匹配级别	匹配策略	置信度	说明
精确匹配	书名完全一致 + 作者完全一致	100%	直接采用，无需人工确认
宽松匹配	书名归一化后一致 + 作者前4字匹配	85%	自动采用，提示用户
模糊匹配	书名包含关系 + 作者相似度 >0.8	60%	需用户确认
候选列表	书名关键词搜索结果	30%	展示列表供用户选择

表 4-1 书源匹配级别与策略

书名归一化处理包括：移除全角/半角差异、移除括号及括号内内容（如"(精校版)"）、移除常见后缀（如"全本"、"完结版"）、移除特殊符号和空格。作者匹配支持笔名别名映射（用户可维护映射表）。相似度计算采用编辑距离算法（Levenshtein Distance），归一化到 [0, 1] 区间。

4.3.2 章节映射算法

章节映射是换源的关键难点。不同书源对同一本书的章节划分可能存在差异：有的书源将"第一章 上"、"第一章 下"合并为一个章节，有的拆分为两个；有的书源包含序章和后记，有的不包含。章节映射算法需要在这些差异下尽可能准确地定位到对应章节。

算法采用"标题相似度 + 序号约束"的双重策略。首先提取当前章节标题，在新书源的章节列表中计算标题相似度（编辑距离 + 关键词重合度）。同时以原章节序号为基准，在正负 5 章的窗口内优先搜索。综合相似度得分和序号接近度，选取最佳匹配。如果最高得分低于阈值（0.5），则提示用户手动选择对应章节。

```
/// 章节映射器
class ChapterMapper {
/// 将旧书源的章节序号映射到新书源的章节序号
/// [oldChapters] 旧书源章节列表
/// [newChapters] 新书源章节列表
/// [currentOrder] 当前阅读章节序号
/// 返回新书源中对应的章节序号
int mapChapter({
  required List oldChapters,
  required List newChapters,
  required int currentOrder,
}) {
  final oldChapter = oldChapters[currentOrder];
  final oldTitle = _normalizeTitle(oldChapter.title);

  // 1. 在序号窗口内搜索 (±5章)
  final windowStart = (currentOrder - 5).clamp(0, newChapters.length - 1);
  final windowEnd = (currentOrder + 5).clamp(0, newChapters.length - 1);
```



```

double bestScore = 0;
int bestIndex = currentOrder.clamp(0, newChapters.length - 1);

for (int i = windowStart; i <= windowEnd; i++) {
    final newTitle = _normalizeTitle(newChapters[i].title);
    final score = _titleSimilarity(oldTitle, newTitle);
    // 序号距离惩罚：每偏离1章扣0.05分
    final penalty = (i - currentOrder).abs() * 0.05;
    final finalScore = score - penalty;
    if (finalScore > bestScore) {
        bestScore = finalScore;
        bestIndex = i;
    }
}

// 2. 如果窗口内未找到匹配，全局搜索
if (bestScore < 0.5) {
    for (int i = 0; i < newChapters.length; i++) {
        final newTitle = _normalizeTitle(newChapters[i].title);
        final score = _titleSimilarity(oldTitle, newTitle);
        if (score > bestScore) {
            bestScore = score;
            bestIndex = i;
        }
    }
}

return bestIndex;
}

double _titleSimilarity(String a, String b) {
    // 编辑距离相似度 + 关键词重合度
    final editSim = 1 - _levenshtein(a, b) /
        max(a.length, b.length, 1);
    final kwSim = _keywordOverlap(a, b);
    return editSim * 0.6 + kwSim * 0.4;
}
}

```

4.4 书源格式设计

书源以 JSON 格式定义，支持导入、导出和分享。书源格式借鉴了业界流行的阅读App书源格式（如开源阅读），但进行了简化和结构化，降低了编写门槛。一个书源 JSON 文件可以包含多个书源定义，便于批量导入。

```

{
    "bookSourceName": "示例文学网",

```

```

"bookSourceUrl": "https://www.example.com",
"bookSourceGroup": "综合",
"bookSourceType": 0,
"bookSourceComment": "示例书源，仅供学习参考",
"enabled": true,
"weight": 50,

"searchUrl": "https://www.example.com/search?q={{key}}&page;={{page}}",
"searchRule": {
"bookList": "css:.result-list > .item",
"name": "css:.title@text",
"author": "css:.author@text##作者: ",
"bookUrl": "css:.title a@href",
"coverUrl": "css:img@src",
"intro": "css:.intro@text",
"category": "css:.tag@text"
},

"detailRule": {
"name": "css:h1.book-title@text",
"author": "css:.author@text",
"intro": "css:.intro@text##简介: ",
"coverUrl": "css:.cover img@src",
"category": "css:.meta .cat@text",
"wordCount": "css:.meta .word@text##字",
"status": "css:.meta .status@text",
"lastChapter": "css:.last-chapter@text",
"chapterListUrl": "css:.chapter-list@href"
},

"chapterListRule": {
"chapterList": "css:li.chapter-item",
"chapterName": "css:a@text",
"chapterUrl": "css:a@href",
"isVip": "css:.vip@text##VIP"
},

"contentRule": {
"content": "css:#content@html",
"nextPageUrl": "css:a.next-page@href",
"replace": [
"请收藏本站.*",
"本章未完.*点击下一页继续"
]
},

"loginUrl": "",
"loginRule": {}
}

```

书源规则采用"CSS选择器 + 正则后处理"的两段式设计。CSS选择器负责定位 HTML 元素，正则表达式负责清洗文本（通过 ## 分隔符附加在选择器后面）。这种设计覆盖了绝大多数网站的内容提取需求，同时保持规则的可读性和可维护性。对于需要 JavaScript 渲染的页面，规则中可声明 "renderType": "webview"，系统将切换到 WebView 渲染模式来获取页面内容。

4.5 多源管理策略

一本书可以同时关联多个书源，系统维护一个"书源优先级队列"。每个关联书源有以下属性：书源ID、匹配置信度、最后可用时间、内容完整度评分、更新延迟评分。系统根据这些属性动态计算综合评分，自动选择最优书源作为"主源"。当主源不可用时，自动降级到次优书源。

评分维度	计算方式	权重
匹配置信度	书源匹配时计算的置信度 (0-1)	30%
内容完整度	完整章节数 / 总章节数	25%
更新及时性	1 - (最后更新延迟天数 / 30)	20%
响应速度	1 - (平均响应时间 / 5000ms)	15%
可用性	成功请求次数 / 总请求次数	10%

表 4-2 书源综合评分维度

综合评分 = 各维度得分 × 权重之和。系统每小时在后台执行一次评分更新，每次网络请求的结果都会更新对应维度的统计数据。当主源综合评分低于阈值（0.5）且持续超过 3 次检查周期时，系统触发自动换源流程，并在下次用户打开应用时提示"已自动切换到更优书源"。

第五章 数据模型设计

5.1 核心实体关系

应用的数据模型围绕"书籍-章节-书源"三个核心实体展开。书籍是用户阅读的主体，章节是书籍的内容单元，书源是章节内容的提供者。一个书籍可以关联多个书源（多源阅读），一个书源可以提供多本书籍。阅读进度、书签、笔记等用户数据挂载在书籍和章节上，与书源无关——这意味着换源不会丢失阅读进度和笔记。

5.2 数据库表结构

以下是用 Drift (SQLite) 定义的数据库表结构。Drift 提供类型安全的 ORM 和响应式查询，适合 Flutter 应用的本地数据管理需求。

5.2.1 书籍表 (books)

字段	类型	说明
id	INTEGER PK	自增主键
title	TEXT NOT NULL	书名
author	TEXT	作者
cover_path	TEXT	本地封面路径
intro	TEXT	简介
category	TEXT	分类
type	TEXT	类型: local / online / hybrid
local_path	TEXT	本地文件路径（本地书）
format	TEXT	格式: txt / epub / pdf
total_chapters	INTEGER	总章节数
word_count	INTEGER	总字数
status	TEXT	连载中 / 已完结
created_at	INTEGER	创建时间戳

字段	类型	说明
updated_at	INTEGER	更新时间戳
group_id	INTEGER	书架分组ID
sort_order	INTEGER	排序序号

表 5-1 books 表结构

5.2.2 书源关联表 (book_sources)

字段	类型	说明
id	INTEGER PK	自增主键
book_id	INTEGER FK	关联书籍ID
source_id	TEXT NOT NULL	书源标识
source_name	TEXT	书源名称
book_key	TEXT	书源内书籍标识
is_primary	INTEGER	是否为主源 (0/1)
confidence	REAL	匹配置信度
score	REAL	综合评分
last_check	INTEGER	最后检查时间
last_available	INTEGER	最后可用时间
chapter_count	INTEGER	该书源章节数
enabled	INTEGER	是否启用 (0/1)

表 5-2 book_sources 表结构（多源关联）

5.2.3 章节表 (chapters)

字段	类型	说明
id	INTEGER PK	自增主键

字段	类型	说明
book_id	INTEGER FK	书籍ID
source_id	TEXT	来源书源ID
chapter_key	TEXT	书源内章节标识
title	TEXT NOT NULL	章节标题
order_index	INTEGER	排序序号
content_path	TEXT	内容文件路径
is_cached	INTEGER	是否已缓存 (0/1)
is_vip	INTEGER	是否VIP (0/1)
word_count	INTEGER	字数
fetches_at	INTEGER	内容获取时间

表 5-3 chapters 表结构

5.2.4 阅读进度表 (read_progress)

字段	类型	说明
book_id	INTEGER PK FK	书籍ID
chapter_index	INTEGER	当前章节序号
page_index	INTEGER	当前页码
char_offset	INTEGER	章节内字符偏移
scroll_offset	REAL	滚动偏移量
reading_time	INTEGER	累计阅读时长(秒)
last_read_at	INTEGER	最后阅读时间
progress_percent	REAL	全书进度百分比

表 5-4 read_progress 表结构

5.2.5 书签与笔记表

表名	字段	说明
bookmarks	id, book_id, chapter_index, char_offset, label, color, created_at	书签标记
notes	id, book_id, chapter_index, start_offset, end_offset, content, note, color, created_at	划线笔记
read_history	id, book_id, chapter_index, action, timestamp	阅读历史轨迹

表 5-5 书签、笔记与历史表

5.3 章节缓存策略

章节内容采用"文件 + 数据库"的混合缓存策略。章节正文以纯文本文件存储在应用沙盒目录下（路径格式：cache/{book_id}/{source_id}/{chapter_key}.txt），数据库仅记录元数据和文件路径。这种设计避免了将大量文本写入数据库导致的性能问题，同时便于缓存清理——删除目录即可释放空间。

缓存采用"预加载 + 滑动窗口"策略。当用户阅读到第 N 章时，系统在后台预加载第 N+1、N+2 章的内容；当用户阅读到第 N+1 章时，预加载窗口滑动到 N+2、N+3，同时第 N-1 章的内容保留在缓存中（支持回退阅读）。缓存的章节总数上限为 50 章，超出时按 LRU 策略淘汰最早的章节。预加载使用 Isolate 在后台执行，不阻塞 UI 线程。

第六章 本地文件与存储方案

6.1 文件格式支持

应用支持四种主流电子书格式的本地导入与解析。每种格式有独立的解析器，解析后统一转换为内部ChapterContent 结构，使阅读引擎无需关心源文件格式。

格式	解析方案	特点
TXT	自研流式解析器	支持编码检测(UTF-8/GBK/Big5)、章节正则识别、大文件分块加载
EPUB	epubx + html2text	支持 EPUB 2.0/3.0、CSS 样式提取、图片资源解压、目录解析
PDF	syncfusion_flutter_pdf viewer	原生渲染、文本提取、书签同步、文字选择
MOBI/AZW3	自研解析器(参考mobitool)	支持 MOBI/KF8 格式、HTML 解析、元数据提取

表 6-1 文件格式支持方案

6.2 TXT 文件解析

TXT 是最常见的本地阅读格式，但也是最需要处理的格式——没有结构化的目录信息，编码不确定，章节划分需要启发式识别。应用的 TXT 解析器采用多阶段处理流程。

编码检测阶段。读取文件前 4KB 字节，使用 ICU 编码检测库判断可能的编码。优先级依次为 UTF-8 BOM、UTF-16 BOM、UTF-8（无BOM）、GBK、Big5。检测结果置信度低于 80% 时，提示用户手动选择编码。

章节识别阶段。使用正则表达式匹配章节标题，支持以下常见格式。匹配到的章节标题作为章节分隔点，将全文切分为多个章节。若全文未匹配到任何章节标题，则按固定字数（默认 10000 字）切分。

```
// 章节标题正则表达式（按优先级排序）
final chapterPatterns = [
  r'^第[一二三四五六七八九十百千零\d]+[章节回卷集部篇话]', // 第一章 / 第1章 / 第一节
  r'^[Chapter|CHAPTER]\s+\d+', // Chapter 1
  r'^第[\d]+章', // 第1章（纯数字）
  r'^[一二三四五六七八九十]+[、.. ]', // 一、 二.
  r'^\(\d+\)', // (1) (2)
  r'^卷[一二三四五六七八九十\d]+', // 卷一 / 卷1
```



```
];
```

大文件处理阶段。对于超过 10MB 的 TXT 文件，解析器采用流式读取 + 增量解析策略。使用 Dart 的 Stream API 分块读取文件（每块 64KB），在 Isolate 中执行章节识别，识别到章节边界后立即写入数据库。这样用户无需等待整个文件解析完成即可开始阅读，解析进度在后台持续推进。

6.3 EPUB 解析

EPUB 本质上是一个 ZIP 压缩包，包含 XHTML 内容文件、CSS 样式、图片资源、目录信息（NCX 或 Nav）。解析器首先解压 ZIP 包到临时目录，读取 META-INF/container.xml 定位 OPF 文件，从 OPF 中解析元数据（书名、作者、封面）和 spine（阅读顺序）。然后按 spine 顺序读取每个 XHTML 文件，使用 html 包解析 HTML，提取纯文本内容，同时保留段落结构用于排版。图片资源解压到缓存目录，在阅读时按需加载。

6.4 本地存储架构

存储类型	用途	容量预估
SQLite (Drift)	结构化数据：书籍、章节元数据、进度、书签、笔记	< 50MB
文件系统 - 书籍	导入的本地书籍文件（TXT/EPUB/PDF）	用户决定
文件系统 - 缓存	在线章节内容缓存、EPUB解压资源、封面图片	< 500MB (可清理)
SharedPreferences	应用配置、阅读设置、主题偏好	< 1MB
Hive Box	书源列表、搜索历史、最近阅读列表	< 5MB

表 6-2 本地存储架构

存储路径规划遵循各平台规范。iOS 使用 App 的 Documents 目录存储用户导入的书籍，Library/Caches 存储缓存数据；Android 使用 app_flutter 目录存储用户数据，cache 缓存目录存储可回收的缓存；桌面端使用用户主目录下的 .reading_app 文件夹。所有路径通过 path_provider 插件获取，不硬编码。

第七章 阅读引擎设计

7.1 阅读模式

阅读引擎支持两种核心阅读模式：翻页模式和滚动模式。翻页模式模拟实体书的阅读体验，内容按屏幕分页，用户通过左右滑动或点击切换页面。滚动模式则像浏览网页一样，内容连续排列，用户上下滚动阅读。两种模式共享同一套排版引擎，仅在渲染和交互层有差异。

特性	翻页模式	滚动模式
交互方式	左右滑动 / 点击翻页	上下滚动
分页计算	需要（预计算每页内容）	不需要（连续渲染）
动画效果	仿真翻页 / 平移 / 渐变	惯性滚动
性能特点	每页渲染轻量，翻页瞬时	长章节渲染较重，滚动流畅
适用场景	小说、文学	技术文档、长篇文章
电量消耗	较低（页面静止时不重绘）	较高（持续滚动重绘）

表 7-1 翻页模式与滚动模式对比

7.2 分页引擎

分页引擎是翻页模式的核心。引擎接收章节文本和排版参数（字体、字号、行距、边距、屏幕尺寸），输出每页应显示的文本范围（起始偏移量和结束偏移量）。分页计算在 Isolate 中执行，避免阻塞 UI 线程。计算结果缓存在内存中，翻页时直接读取，无需重新计算。

```
/// 分页引擎
class PaginationEngine {
  /// 计算章节分页
  /// [text] 章节全文
  /// [config] 排版配置
  /// [canvasSize] 渲染区域尺寸
  /// 返回每页的字符偏移量列表
  Future< > paginate({
    required String text,
    required LayoutConfig config,
    required Size canvasSize,
  }) async {
    return await Isolate.run(() {
      final pages = [];
      int offset = 0;
```

```

final metrics = _buildTextMetrics(config);

while (offset < text.length) {
  final pageEnd = _findPageBreak(
    text, offset, metrics, canvasSize,
  );
  pages.add(PageRange(start: offset, end: pageEnd));
  offset = pageEnd;
}

return pages;
});
}

/// 查找页边界：从 offset 开始，逐行测量直到填满页面
int _findPageBreak(
  String text, int start,
  TextMetrics metrics, Size canvasSize,
) {
  double usedHeight = 0;
  double lineWidth = canvasSize.width;
  int pos = start;

  while (pos < text.length && usedHeight < canvasSize.height) {
    // 1. 找到当前行结束位置（换行符或自动换行）
    final lineEnd = _findLineEnd(text, pos, metrics, lineWidth);
    // 2. 计算行高
    usedHeight += metrics.lineHeight;
    // 3. 段落间距
    if (lineEnd < text.length && text[lineEnd] == '\n') {
      usedHeight += metrics.paragraphSpacing;
    }
    pos = lineEnd;
  }

  // 回退到最后一个完整行
  while (pos > start && pos < text.length &&
    text[pos] != '\n' &&
    !_isChinesePunctuation(text[pos - 1])) {
    pos--;
  }
  return pos;
}
}

```

7.3 排版引擎

排版引擎负责将章节文本转换为可渲染的富文本布局。引擎支持以下排版特性：中英文混排自动加间距、标点避头尾（行首避头、行尾避尾）、段落首行缩进、多级标题样式、引文块样式、分隔线、注音标注。排版参数全部可配置，用户可以自定义字体、字号、行距、段距、边距、文字颜色、背景颜色等。

排版参数	默认值	可调范围
字体	系统默认衬线	系统字体 / 自定义TTF / 思源宋体 / 思源黑体
字号	18 sp	12 - 32 sp
行距	1.6 倍	1.2 - 2.5 倍
段距	0.8 倍行高	0 - 2 倍行高
页边距	24 dp	8 - 48 dp
首行缩进	2 字符	0 - 4 字符
文字颜色	#333333	任意颜色
背景颜色	#F5F5F0	任意颜色

表 7-2 排版参数配置

7.4 主题系统

主题系统支持日间、夜间、护眼、墨水屏四种预设主题，以及无限自定义主题。每个主题定义背景色、文字色、强调色、链接色、选中高亮色五个核心颜色。夜间模式不仅切换颜色，还支持降低亮度和对比度以减轻眼睛疲劳。护眼模式采用暖色调（色温约 3000K），适合长时间阅读。墨水屏模式采用高对比度的黑白配色，专为 E-ink 设备优化。

主题	背景色	文字色	适用场景
日间	#F5F5F0	#333333	白天室内阅读
夜间	#1A1A1A	#B0B0B0	暗光环境阅读
护眼	#F0E6D2	#5B4636	长时间阅读
墨水屏	#FFFFFF	#000000	E-ink 设备
自定义	用户定义	用户定义	个性化需求

表 7-3 预设主题配色

第八章 UI/UX设计

8.1 页面结构

应用采用底部导航 + 堆栈页面的标准结构。底部导航包含四个入口：书架、书源、发现、我的。书架页展示用户的书籍列表，支持网格和列表两种视图；书源页管理已导入的书源；发现页展示书源推荐内容和分类浏览；我的页包含设置、阅读统计、云同步等功能。阅读页是一个全屏的沉浸式页面，覆盖底部导航栏，通过手势返回退出。

页面	功能	交互要点
书架页	书籍展示、分组、排序、搜索	长按多选、拖拽排序、下拉刷新
搜索页	全平台搜索、结果聚合	多书源并发搜索、结果去重、来源标识
书籍详情页	书籍信息、章节目录、换源	简介展开收起、目录加载、换源入口
阅读页	章节内容阅读	点击中间区域呼出菜单、左右滑动翻页
阅读设置页	排版参数调节	实时预览、字体导入、主题选择
书源管理页	书源列表、导入、编辑	批量导入、编辑验证、排序分组
书源编辑页	书源规则编辑、调试	规则验证、实时预览结果

表 8-1 核心页面结构

8.2 阅读页交互设计

阅读页是用户使用频率最高的页面，交互设计需要兼顾沉浸感和功能可达性。阅读页默认全屏显示，仅展示文本内容，所有功能控件隐藏。用户点击屏幕中间区域（约占屏幕宽度的 40%、高度的 50%）呼出顶部工具栏和底部菜单栏；点击左侧区域翻到上一页，点击右侧区域翻到下一页。这种设计源自经典阅读App的交互模式，已被广泛验证为最优方案。

顶部工具栏包含返回按钮、书名、换源按钮和更多操作（书签、笔记、分享）。底部菜单栏包含四个入口：目录、进度、设置、换源。底部菜单采用 BottomSheet 形式展开，不遮挡阅读内容。阅读设置面板支持实时预览——用户调节字号、行距等参数时，背景阅读内容实时更新效果。

8.3 换源交互流程

换源入口位于阅读页底部菜单栏和顶部工具栏。用户点击"换源"后，底部弹出换源面板，面板展示当前书籍关联的所有书源列表，按综合评分排序。每个书源条目显示书源名称、章节数、最后更新时间、综合评分星级。当前使用的书源高亮标记。

用户选择新书源后，系统执行换源流程：显示加载动画 → 获取新书源章节目录 → 映射当前阅读位置 → 加载对应章节内容 → 切换完成。整个过程通常在 2-5 秒内完成。如果章节映射置信度较低，系统在切换后弹出提示"已切换到 XX 书源，当前章节可能与之前略有差异，请在目录中确认位置"。用户可以随时撤回换源操作，恢复到之前的书源。

8.4 搜索体验设计

搜索是发现书籍的主要入口。用户输入关键词后，系统在所有已启用的书源中并发搜索，结果按书源分组展示，同时提供"合并去重"视图。合并视图通过书名+作者去重，同一本书的不同书源结果合并为一条，展开后可以看到各书源的信息差异（章节数、更新状态等）。搜索结果支持按书名、作者、书源、更新时间排序。搜索进度以进度条形式展示（已完成书源数 / 总书源数），用户可以提前查看已返回的结果，无需等待所有书源搜索完成。

第九章 网络层与书源引擎

9.1 书源执行引擎

书源执行引擎（SourceEngine）是连接书源定义和实际网络请求的桥梁。引擎接收书源规则和输入参数，执行"构建URL → 发送请求 → 解析响应 → 提取数据 → 清洗内容"的完整流程，返回结构化结果。引擎的设计需要处理多种异常场景：网络超时、HTML 结构变更、反爬机制、编码问题等。

```
/// 书源执行引擎
class SourceEngine {
    final Dio _client;

    /// 执行搜索
    Future< executeSearch({
        required BookSource source,
        required String keyword,
        int page = 1,
    }) async {
        // 1. 构建搜索URL (模板替换)
        final url = _buildUrl(
            source.searchUrl,
            {'key': keyword, 'page': page.toString()},
        );

        // 2. 发送请求
        final html = await _fetchHtml(url, source);

        // 3. 解析HTML
        final doc = html_parser.parse(html);

        // 4. 应用搜索规则提取数据
        final list = _selectElements(doc, source.searchRule.bookList);
        final results = list.map((el) => SearchResult(
            bookKey: _extract(el, source.searchRule.bookUrl),
            title: _extract(el, source.searchRule.name),
            author: _extract(el, source.searchRule.author),
            coverUrl: _extract(el, source.searchRule.coverUrl),
            intro: _extract(el, source.searchRule.intro),
            sourceId: source.id,
        )).where((r) => r.title.isNotEmpty).toList();

        return results;
    }

    /// 获取章节内容 (含分页处理)
    Future< executeContent({
        required BookSource source,
        required String bookKey,
```

```

required String chapterKey,
}) async {
final url = _buildUrl(chapterKey, {});
final html = await _fetchHtml(url, source);
final doc = html_parser.parse(html);

var content = _extract(doc, source.contentRule.content);

// 处理内容分页
var nextUrl = _extract(doc, source.contentRule.nextPageUrl);
while (nextUrl.isNotEmpty) {
final nextHtml = await _fetchHtml(nextUrl, source);
final nextDoc = html_parser.parse(nextHtml);
content += _extract(nextDoc, source.contentRule.content);
nextUrl = _extract(nextDoc, source.contentRule.nextPageUrl);
}

// 应用正则替换规则（广告清洗）
for (final pattern in source.contentRule.replace) {
content = content.replaceAll(
RegExp(pattern, multiline: true), '',
);
}

return ChapterContent(
chapterKey: chapterKey,
title: _extract(doc, source.detailRule.name),
content: content,
sourceId: source.id,
fetchedAt: DateTime.now(),
);
}
}

```

9.2 并发控制

多书源搜索需要并发控制来平衡速度和稳定性。引擎采用"分组并发 + 限速"策略。所有已启用书源按域名分组，同一域名的请求串行执行（避免触发反爬），不同域名并行执行。全局并发数限制为 10（可配置），超出部分排队等待。每个请求设置 10 秒超时，超时后标记书源为"响应缓慢"并继续等待其他书源返回。

参数	默认值	说明
全局并发数	10	同时进行的网络请求数上限
单源并发数	1	同一书源的并发请求限制

参数	默认值	说明
请求超时	10秒	单次请求超时时间
重试次数	2	失败后重试次数
重试间隔	1秒	重试前等待时间
搜索总超时	30秒	搜索结果汇总的最大等待时间

表 9-1 并发控制参数

9.3 反爬策略

部分书源网站部署了反爬机制，引擎需要应对常见的反爬策略。对于 User-Agent 检测，引擎默认使用移动端浏览器 UA，并支持在书源规则中自定义。对于 Referer 检测，引擎自动将书源 baseUrl 设为 Referer。对于 Cookie 验证，引擎维护每个书源的 Cookie Jar，自动管理会话。对于 JavaScript 渲染需求，引擎支持切换到 WebView 模式——使用平台 WebView 加载页面，等待渲染完成后提取 HTML。

9.4 错误处理与降级

网络请求不可避免会遇到各种错误。引擎采用分层错误处理策略。请求层错误（超时、DNS失败、连接拒绝）自动重试 2 次，重试间隔递增。解析层错误（选择器未匹配、字段为空）记录日志并返回空结果，不中断整体流程。业务层错误（书源规则过期、页面结构变更）标记书源为"需要更新"，并在书源管理页提示用户。当某书源连续 5 次请求失败时，自动禁用该书源并通知用户。

在换源场景下，错误处理需要更加优雅。当当前书源请求失败时，系统自动尝试切换到次优书源，整个过程对用户透明——仅在切换成功后显示一条轻提示"已切换到 XX 书源"。如果所有关联书源都不可用，系统提示用户检查网络或添加更多书源。

第十章 实现路线图

10.1 阶段划分

项目按三个阶段递进实现，每个阶段交付可用的产品版本，逐步完善功能体系。

阶段	周期	交付目标	核心功能
MVP	第1-6周	本地阅读可用版	TXT导入、书架、基础阅读引擎、日/夜主题
V1.0	第7-14周	书源系统可用版	书源导入、搜索、在线阅读、换源机制
V2.0	第15-22周	全功能版	EPUB/PDF支持、云同步、TTS、高级排版

表 10-1 实现阶段划分

10.2 MVP 阶段（第1-6周）

MVP 阶段聚焦于本地阅读体验的打磨，确保核心阅读功能稳定可用。第 1-2 周：搭建项目骨架——Flutter 项目初始化、依赖配置、四层架构搭建、数据库 Schema 设计与 Drift 代码生成、路由配置。第 3-4 周：实现 TXT 文件导入与解析——编码检测、章节识别、大文件流式解析。搭建书架页面——书籍列表、分组、排序、搜索。第 5-6 周：实现阅读引擎——分页计算、排版渲染、翻页动画、阅读设置面板。实现日间/夜间主题切换。阅读进度保存与恢复。

10.3 V1.0 阶段（第7-14周）

V1.0 阶段实现书源系统和换源机制——应用的差异化核心。第 7-8 周：书源格式设计、书源管理页面、书源导入（JSON 文件、二维码、剪贴板）。书源执行引擎——URL 构建、HTML 请求、规则解析、内容清洗。第 9-10 周：多书源并发搜索、搜索结果聚合去重、书籍详情页。在线章节获取与缓存策略。第 11-12 周：换源机制——书源匹配算法、章节映射算法、多源管理、综合评分。换源交互流程——换源面板、自动换源、换源提示。第 13-14 周：全链路联调、性能优化、异常处理加固。内测发布，收集用户反馈。

10.4 V2.0 阶段（第15-22周）

V2.0 阶段扩展格式支持、同步能力和高级功能。第 15-16 周：EPUB 格式解析与渲染——ZIP 解压、OPF 解析、HTML 内容提取、CSS 样式支持。PDF 渲染集成——基于 syncfusion_flutter_pdfviewer 的阅读体验。第 17-18 周：云同步功能——基于自建后端或第三方 BaaS（如 Supabase）的同步服务。阅读进度、书源、书签的增量同步。多设备冲突解决

策略。第 19-20 周：TTS 朗读——集成系统 TTS 引擎，支持语音选择、语速调节、自动翻页。高级排版——标点避头尾、多级标题、注音支持。第 21-22 周：桌面端适配——响应式布局、键盘快捷键、文件拖拽导入。HarmonyOS 适配验证。正式发布。

10.5 风险与应对

风险	影响	应对措施
书源网站反爬升级	书源失效，用户无法获取内容	WebView渲染模式、多源备份、社区书源更新
HarmonyOS Flutter适配延迟	无法覆盖鸿蒙用户	优先保证iOS/Android，鸿蒙延后支持
EPUB/PDF渲染性能不足	大文件阅读卡顿	Isolate异步渲染、分块加载、内存监控
云同步服务成本	运营压力	MVP阶段不做云同步，V2.0引入时优先用BaaS
书源格式兼容性	用户导入书源失败	兼容主流书源格式、提供书源编辑器、验证工具

表 10-2 项目风险与应对措施